Análisis Exhaustivo de la Estabilidad Numérica en la Arquitectura Mamba: Gradientes, Precisión y Optimizaciones de Kernel

Introducción al Paradigma de los Modelos de Espacio de Estados Selectivos

El panorama contemporáneo del aprendizaje profundo ha estado dominado de manera incontrovertible por la arquitectura Transformer, cuyo mecanismo de autoatención global ha permitido un modelado de secuencias con una eficacia sin precedentes. Sin embargo, el éxito empírico de los Transformers está intrínsecamente ligado a una limitación algorítmica insalvable: su complejidad computacional y sus requerimientos de memoria escalan cuadráticamente con respecto a la longitud de la secuencia de entrada.[1] Esta barrera ha impulsado el desarrollo de arquitecturas subcuadráticas, entre las cuales los Modelos de Espacio de Estados (State Space Models, SSMs) han emergido como la frontera de investigación más prometedora.[2] Dentro de esta familia, la arquitectura Mamba representa un salto evolutivo monumental al combinar la eficiencia computacional lineal con una capacidad de razonamiento dependiente del contexto que rivaliza con los mejores modelos basados en atención.[2]

Los modelos de espacio de estados tradicionales, estructurados bajo el marco fundamental de sistemas invariantes en el tiempo (Linear Time-Invariant, LTI), como la arquitectura S4 (Structured State Space for Sequences), logran una eficiencia notable mediante el uso de convoluciones globales posibilitadas por la Transformada Rápida de Fourier (FFT).[5] No obstante, estos sistemas invariantes procesan cada token de entrada con la misma dinámica rígida, operando de manera análoga a una línea de ensamblaje inalterable.[7] Esta incapacidad para realizar razonamiento basado en el contenido provoca que los modelos LTI fallen en tareas que exigen la selección dinámica de información, como la evaluación de contextos en lenguaje natural o el seguimiento de estados complejos.[1]

Mamba resuelve esta deficiencia introduciendo un mecanismo de selección audaz: permite que los parámetros del espacio de estados sean funciones dependientes del input, sacrificando deliberadamente la invarianza temporal.[2] Esta modificación metodológica transforma el modelo en una red recurrente dinámica que puede retener información crítica y olvidar selectivamente el ruido de fondo, logrando un rendimiento superior en modelado de lenguaje, genómica y series temporales.[2] El parámetro central que orquesta esta selección es

Δ (delta), el cual actúa como un paso de discretización adaptable que modula el flujo de información hacia la memoria latente del modelo.[5]

Sin embargo, el abandono de la convolución global condena al modelo a ser evaluado en modo recurrente secuencial.[5] Al desencadenar una propagación autorregresiva a través de secuencias que pueden superar el millón de tokens, la arquitectura enfrenta desafíos de estabilidad numérica colosales. La retropropagación a través del tiempo (Backpropagation Through Time, BPTT) para calcular los gradientes de Δ y de la matriz de transición A revela una susceptibilidad extrema al crecimiento exponencial de derivadas y a la propagación catastrófica de errores de redondeo.[9] Al operar bajo la precisión simple de 32 bits (FP32) o precisiones mixtas dentro del hardware de las Unidades de Procesamiento Gráfico (GPU), estos gradientes exhiben inestabilidades que amenazan con destruir el entrenamiento del modelo.[6] Este informe investiga exhaustivamente las raíces matemáticas de estas anomalías numéricas y detalla los sofisticados estabilizadores analíticos y optimizaciones de bajo nivel en el kernel de C++/CUDA que permiten a Mamba escalar y converger con éxito empírico.

Fundamentos Matemáticos: De Sistemas Continuos a Espacios Discretos

Para diagnosticar las vulnerabilidades de los gradientes en la arquitectura Mamba, es preceptivo establecer una base teórica inquebrantable de la formulación de los Modelos de Espacio de Estados. La premisa fundamental de un SSM es el mapeo de una señal de entrada unidimensional continua $x(t) \in \mathbb{R}$ hacia una señal de salida $y(t) \in \mathbb{R}$, mediado por la evolución de un estado latente u oculto de alta dimensionalidad $h(t) \in \mathbb{R}^N$.[13] Las dinámicas que rigen este sistema se describen mediante un sistema de ecuaciones diferenciales ordinarias (EDOs) de primer orden:

$$h'(t) = Ah(t) + Bx(t)$$

$$y(t) = Ch(t) + Dx(t)$$

En esta formulación, la matriz $A \in \mathbb{R}^{N \times N}$ encapsula la dinámica de transición del estado latente, determinando de qué manera la información pasada se propaga o decae a través del tiempo.[13] El vector $B \in \mathbb{R}^{N \times 1}$ dicta la proyección de la entrada actual hacia el espacio de estados, mientras que el vector $C \in \mathbb{R}^{1 \times N}$ orquesta la transformación del estado latente nuevamente hacia el dominio de la predicción de salida. Finalmente, D funciona como una

conexión residual o "skip connection" directa desde la entrada hasta la salida.[14]

Dado que los sistemas de aprendizaje profundo computan datos digitales inherentes al procesamiento de secuencias, como texto tokenizado o señales de audio muestreadas, el sistema continuo debe ser inexorablemente convertido a una representación discreta. Esta transformación se denomina discretización y se realiza mediante la introducción de un parámetro de tamaño de paso denotado como Δ.[3]

El Método del Retenedor de Orden Cero (Zero-Order Hold)

La técnica matemática predilecta en la arquitectura Mamba para ejecutar esta discretización es el Retenedor de Orden Cero (Zero-Order Hold o ZOH). El principio del ZOH asume que la señal de entrada $x(t)$ se mantiene constante durante el intervalo de muestreo $[t_k, t_{k+1}]$, cuya longitud está dictada por Δ.[5] Al integrar analíticamente la ecuación diferencial del estado bajo esta asunción de constancia, los parámetros continuos (A, B) se transforman en sus homólogos discretos $(\bar{A}, \bar{B})$ de acuerdo a las siguientes soluciones algebraicas exactas:

$$\bar{A} = \exp(\Delta A)$$

$$\bar{B} = (\Delta A)^{-1}(\exp(\Delta A) - I)\Delta B$$

Donde I representa la matriz identidad y $\exp$ denota la exponencial de una matriz.[16] Para eludir el enorme costo computacional y la inestabilidad que supone calcular la inversa matricial $(\Delta A)^{-1}$, los autores de Mamba emplean una deducción pragmática. Argumentan que, dado que el diseño del modelo empuja empíricamente a los valores diagonales de ΔA a ser dimensionalmente minúsculos, la expansión en series de Taylor de primer orden reduce la formulación.[16] La primera aproximación de Taylor dictamina que $\exp(\Delta A) \approx I + \Delta A$. Sustituyendo esto en la ecuación discreta original, se obtiene el Método de Euler para la matriz de inyección de entrada:

$$\bar{B} = (\Delta A)^{-1}(I + \Delta A - I)\Delta B \approx \Delta B$$

Esta deducción es crítica para las implementaciones eficientes de Mamba. Sin embargo, la matriz de evolución del estado, $\bar{A}$, retiene estrictamente su formulación exponencial completa ($\bar{A} = \exp(\Delta A)$), ya que capturar el decaimiento exacto de la memoria a lo largo del tiempo

es un aspecto innegociable para la capacidad de retención de largo plazo.[16]

Con los tensores ya discretizados, el sistema continuo se traduce a un par de ecuaciones recurrentes de diferencias finitas, análogas a la estructura fundamental de una Red Neuronal Recurrente (RNN) convencional:

$$h_t = \bar{A}_t h_{t-1} + \bar{B}_t x_t$$

$$y_t = C_t h_t + D x_t$$

La Selectividad: Δ como Función del Input

En arquitecturas invariantes como S4, los parámetros Δ, A, B y C son aprendidos durante el entrenamiento, pero se mantienen estáticos y constantes para todos los tokens durante la fase de inferencia.[5] Esta rigidez significa que un pronombre trivial como "él" recibe exactamente el mismo trato dinámico en el estado oculto que un sustantivo altamente informativo.[3]

La genialidad paradigmática de Mamba es abandonar la invarianza LTI. En su lugar, el tamaño de paso Δ_t, junto con las proyecciones B_t y C_t, se convierten en funciones paramétricas directas de la entrada actual x_t.[5] Intuitivamente, y como es ampliamente discutido en la literatura de la arquitectura, Δ_t adquiere la función análoga a la de una compuerta o válvula en un mecanismo de Long Short-Term Memory (LSTM).[5]

Si la red proyecta un Δ_t pequeño (cercano a cero) para el token en curso, entonces $\Delta_t A \approx 0$, y por ende $\bar{A}_t = \exp(\Delta_t A) \approx I$. De manera recíproca, $\bar{B}_t = \Delta_t B \approx 0$. Esto resulta en $h_t \approx I \cdot h_{t-1} + 0$, indicando empíricamente que el modelo ignora la entrada presente y preserva incólume el estado de su memoria histórica.[3] Si, por el contrario, el token requiere asimilación agresiva, un Δ_t de alta magnitud mitigará la influencia del pasado (a medida que los componentes negativos de A inducen un rápido decaimiento hacia cero en la función exponencial) y maximizará la sobreescritura con el nuevo contenido provisto por x_t.[3] Esta selectividad dependiente del tiempo dota al SSM de la misma riqueza expresiva que los Transformers, pero introduce una no-linealidad implícita a través de la topología multiplicativa de sus matrices variantes a lo largo de la longitud temporal L.

Derivación Analítica de Gradientes: El Problema de la Explosión Exponencial

La flexibilidad ganada por la inclusión de secuencias variantes de Δ_t y matrices discretizadas destruye la capacidad de expresar el cómputo secuencial como un solo núcleo convolucional global, forzando a Mamba a depender de la recurrencia estricta para resolver sus pasajes iterativos.[5] En el entrenamiento de arquitecturas profundas, el descenso de gradiente es orquestado por el proceso iterativo de la regla de la cadena (Backpropagation).[18] Las derivadas parciales de la función de pérdida empírica $\mathcal{L}$ con respecto a los tensores internos miden cómo ajustar los parámetros para reducir la entropía de las predicciones.[21]

Sin embargo, retropropagar los gradientes a través de un espacio recurrente autorregresivo es un proceso inherentemente hostil a la estabilidad matemática.[10] Desarrollando el estado recurrente h_t mediante la iteración inversa de sus propios términos a través de los instantes anteriores de tiempo, obtenemos una expresión polinomial densa de productos matriciales cruzados [17]:

$$h_t = \bar{A}_t \bar{A}_{t-1} h_{t-2} + \bar{A}_t \bar{B}_{t-1} x_{t-1} + \bar{B}_t x_t$$

$$h_t = \left(\prod_{i=1}^{t} \bar{A}_i \right) h_0 + \sum_{i=1}^{t} \left(\prod_{j=i+1}^{t} \bar{A}_j \right) \bar{B}_i x_i$$

Para calcular el gradiente de un parámetro dependiente del tiempo particular, como el tamaño del paso Δ_k en el instante arbitrario k, el algoritmo de BPTT debe propagar el gradiente final $\frac{\partial \mathcal{L}}{\partial h_t}$ a través de cada matriz transitoria intermedia $\bar{A}_j$ en forma de multiplicaciones vector-matriz sucesivas.[23] Evaluando matemáticamente este trayecto:

$$\frac{\partial \mathcal{L}}{\partial \Delta_k} = \sum_{t=k}^{L} \frac{\partial \mathcal{L}}{\partial h_t} \frac{\partial h_t}{\partial h_k} \frac{\partial h_k}{\partial \Delta_k}$$

Donde el término clave responsable de comunicar la sensibilidad a largo plazo es el Jacobiano del estado con respecto a los estados anteriores, el cual se condensa como un producto continuo intermedio [10]:

$$\frac{\partial h_t}{\partial h_k} = \prod_{j=k+1}^{t} \bar{A}_j = \prod_{j=k+1}^{t} \exp(\Delta_j A) = \exp\left(\sum_{j=k+1}^{t} \Delta_j A\right)$$

Dado que la matriz $\bar{A}_k = \exp(\Delta_k A)$, la diferenciación local revela que la gradiente parcial es:

$$\frac{\partial \bar{A}_k}{\partial \Delta_k} = A \exp(\Delta_k A)$$

La volatilidad emerge drásticamente de la formulación $\exp(\sum \Delta_j A)$. Si la suma de las proyecciones en el exponente es marginalmente superior a cero, la exponencial causará que el valor explote a un ritmo vertiginoso conforme la longitud de la secuencia L crece, conduciendo a lo que se conoce como *Exploding Gradients* o gradientes explosivos.[9] Este fenómeno resulta en actualizaciones de peso infinitas (NaN o Not a Number), lo cual aborta inmediatamente el bucle de optimización. Inversamente, si la suma escalar es fuertemente negativa, la derivada descenderá geométricamente a ceros absolutos a una velocidad implacable (*Vanishing Gradients*), despojando al modelo de cualquier señal de corrección para dependencias distantes y paralizando su capacidad de memorización.[9]

La Vulnerabilidad Concreta de dt_proj

Empíricamente, los implementadores de Mamba han observado repetidamente la desaparición prematura de gradientes, particularmente focalizados en los parámetros estructurales de la capa de proyección discreta Δ , conocida habitualmente en el código fuente de PyTorch como dt_proj.[24]

En la arquitectura interna, Δ no se inicializa como una constante independiente, sino que se proyecta dinámicamente desde el tensor de entrada mediante una transformación lineal de rango bajo (low-rank). La capa dt_proj mapea típicamente un rango contraído dimensionalmente (e.g., $D_{model}/16$) hacia una dimensión brutalmente expandida a nivel de canal (e.g., $2 \times D_{model}$).[24] Esta expansión masiva (que frecuentemente alcanza proporciones de escalado de $32\times$) exacerba la inestabilidad de la varianza. Una pequeña perturbación durante el flujo reverso BPTT se magnifica al propagarse por una matriz de tan alta divergencia, llevando la salida proyectada y sus correspondientes sesgos (bias) hacia valores extremos. Cuando estas estimaciones aberrantes son insertadas en el tensor exponencial continuo, los gradientes desaparecen progresivamente a lo largo de las capas sucesivas de la red.[24]

Dinámica de la Precisión en Hardware GPU y Limitaciones Estructurales

La inestabilidad teórica dictada por el marco analítico multiplicativo colisiona fatalmente con las tolerancias finitas de la arquitectura subyacente del hardware GPU.[26] El entrenamiento de grandes modelos de lenguaje (LLMs) está fuertemente limitado por la memoria de la Unidad de Procesamiento Gráfico, induciendo al uso de representaciones de menor precisión de coma flotante para escalar de manera viable.

Formatos IEEE 754, FP32, y el Épsilon de la Máquina

Los datos procesados por arquitecturas Mamba en entornos empresariales fluyen a través de representaciones fraccionarias predefinidas. En la precisión doble (FP64), un valor disfruta de 11 bits de exponente y 52 bits de mantisa (una fracción que controla la resolución del dígito), proporcionando entre 15 a 17 cifras decimales precisas.[26] Sin embargo, utilizar FP64 resulta insoportablemente costoso computacionalmente. El advenimiento del aprendizaje profundo moderno instauró la precisión simple FP32 como el estándar tradicional, conformada por 8 bits de exponente y 23 bits de mantisa (~7 dígitos de precisión decimal).[12]

La métrica vital en el análisis numérico de hardware es el "épsilon de la máquina" (ϵ), definido como la distancia formal más pequeña permisible entre el número 1.0 y el número más cercano representable superior a 1.0. Para FP32, $\epsilon \approx 1.19 \times 10^{-7}$.[28] Para formatos de precisión media utilizados agresivamente en clústeres optimizados, como FP16 o BFloat16 (BF16), el épsilon es drásticamente más severo: $\epsilon \approx 9.76 \times 10^{-4}$ y $\epsilon \approx 7.8 \times 10^{-3}$, respectivamente, lo que otorga en el mejor de los casos 2 a 3 cifras de precisión decimal resoluble.[28]

Formato de Precisión	Tamaño Total (Bits)	Bits de Exponente	Bits de Mantisa	Precisión Decimal Estimada	Consecuencia en Evaluación Exponencial Recursiva
FP64 (Doble)	64	11	52	~15 a 17 dígitos	Estabilidad excepcional, pero ancho de

					banda limitante.[26]
FP32 (Simple)	32	8	23	~7 dígitos	Vulnerable a pérdidas de significancia geométrica en $\exp(x)$.[12]
TF32 (Tensor)	19	8	10	~3 a 4 dígitos	Descartamiento agresivo en las multiplicaciones encadenadas de Mamba.[12]
FP16 (Media)	16	5	10	~3 a 4 dígitos	Peligro constante de desbordamiento aritmético (Overflow) al exceder $\pm 65,50$.[29]
BF16 (Bfloat)	16	8	7	~2 a 3 dígitos	Resolución de mantisa paralizante; destruye las variaciones de h_t minúsculas.

					28

Errores de Redondeo en Multiplicaciones Geométricas

El talón de Aquiles de Mamba radica en su recurrencia matricial. Cuando la matriz A condiciona un decaimiento para conservar la memoria del sistema, el resultado de $\bar{A} = \exp(\Delta_t A)$ es frecuentemente un número microscópicamente cercano, pero estrictamente inferior a, 1.0 (por ejemplo, 0.999995). Multiplicar 0.999995 por sí mismo de forma iterativa cientos de miles de veces (para secuencias con longitudes extendidas de hasta 1 millón de tokens) empuja implacablemente a la aritmética de coma flotante más allá del umbral de resolución de su épsilon representable.[28]

El hardware GPU no retiene un historial continuo infinito; por lo tanto, trunca y redondea cada producto provisional al número representable más cercano.[28] En los flujos vectorizados de precisión FP32, estas acumulaciones iterativas resultan frecuentemente en un "underflow" (subdesbordamiento) del estado oculto, donde la acumulación matemática fuerza al registro a redondear violentamente el tensor a 0.0, matando instantáneamente la retropropagación de memoria dependiente en un abismo numérico irrecuperable.[12]

El problema se exacerba aún más en entrenamientos con precisión mixta utilizando *Tensor Cores* (núcleos tensoriales dedicados en arquitecturas como NVIDIA Volta, Ampere y Hopper).[30] Investigaciones experimentales utilizando matrices de Hurwitz y el análisis avanzado de exponentes de Lyapunov han demostrado que la introducción de precisiones microscópicas inferiores inyecta ruido estocástico masivo. Este ruido induce picos de desviación y bifurcaciones irreversibles de divergencia, causando una desestabilización catastrófica del estado del modelo e invalidando por completo la convergencia de las secuencias.[6]

Estabilizadores Matemáticos a Nivel Arquitectónico

Para asegurar que los gradientes no se disparen hacia el infinito debido al crecimiento geométrico descontrolado de sus exponenciales iterativas, la arquitectura de Mamba recurre a robustos principios extraídos de las matemáticas del control dinámico, incorporando estabilizadores estructurales antes de que las operaciones se evalúen en el hardware.[16]

La Reparametrización de la Matriz A y el Espectro Negativo

La principal vacuna analítica en la arquitectura reside en la inicialización y manipulación de la matriz fundamental A. Para asegurar que el producto iterativo de $\bar{A}_j = \exp(\Delta_j A)$ no conduzca a derivadas infinitamente en crecimiento a través del tiempo, Mamba exige que el comportamiento de sus valores propios se mantenga constreñido. Específicamente, el

espectro (el conjunto de valores propios de una matriz) de la transición A debe permanecer confinado matemáticamente en el semiplano negativo estricto.[35]

En la literatura de los SSMs de largo alcance, se comprende que las matrices con valores propios en el semiplano negativo facilitan modelos estables frente a dependencias extendidas (por ejemplo, fundamentado en la teoría clásica de proyección de polinomios de Legendre, HiPPO, y S4D-Real).[37] Si todos los valores de A son estrictamente negativos (< 0), y considerando que Δ es rigurosamente forzado a ser un valor positivo mediante otras medidas de estabilización, el término producto ΔA resulta univariadamente en un dominio denotado exclusivamente por valores menores o iguales a cero.

Bajo la ley de la función exponencial euclidiana, $\exp(x)$ evaluada para todo $x \leq 0$ está inexorablemente anclada al rango acotado de valores $(0, 1]$.[16] Este límite formal elimina el riesgo fundacional de los gradientes explosivos, limitando la magnitud de las memorias iterativas para comportarse exclusivamente de manera disipativa (decadente). En vez de que las derivadas exploten, la retropropagación en reversa es domada, estabilizando radicalmente el proceso.

No obstante, enmarcar esta negatividad espectral durante la optimización no supervisada mediante Descenso de Gradiente Estocástico (SGD) es complejo; los ajustes optimizados comúnmente obligarían a los pesos de la red a oscilar sobre cero, entrando en el dominio positivo y reiniciando el caos exponencial. Para inmunizar la matriz contra esta deriva paramétrica, los desarrolladores de Mamba introdujeron una elegante reparametrización matemática logarítmica para A en el código base de PyTorch.[16]

El modelo no entrena directamente una matriz final de proyección A, sino que define un tensor parametrizado libre e inconmensurable denotado empíricamente como $A_{\log}$.[34] Durante la fase de propagación hacia adelante (forward pass), la matriz de transición empírica materializada es generada mediante la transformación proyectiva:

$$A = -\exp(A_{\log})$$

Donde $A_{\log}$ es almacenado implacablemente bajo el dictado numérico estricto de FP32 y abstraído del proceso de decaimiento de pesos (weight_decay = 0) para resguardar su integridad analítica.[34] Dado que el rango analítico de la función exponencial cubre exclusivamente los números reales positivos ($e^x > 0 \ \forall x \in \mathbb{R}$), la inyección explícita del

signo negativo garantiza lógicamente que A poseerá siempre entradas estrictamente menores que cero, conservando la arquitectura blindada contra perturbaciones cataclísmicas sin entorpecer el proceso de exploración paramétrica ilimitada del espacio optimizable por SGD.[16]

Activación Softplus y Truncamiento de Precisión en Δ

Mientras que la parametrización reprime eficazmente el espectro de la matriz A, la dinámica del salto de discretización Δ sigue siendo impulsada por las impredecibles transformaciones estocásticas producidas por su capa de control dt_proj. Para evadir oscilaciones que generen saltos paramétricos violentos o valores negativos, se implementan no linealidades asintóticas controladas.[25]

En su inicialización formal, las proyecciones asociadas al paso de discretización son amortiguadas implementando la función de activación convexa softplus a nivel de componente. Definida rigurosamente como $\mathrm{softplus}(x) = \ln(1 + \exp(x))$, esta proyección restringe coercitivamente el tensor de entrada de Δ_t hacia el dominio puramente positivo real y provee derivadas progresivamente suaves que limitan la reactividad exponencial destructiva.[41]

Simultáneamente, experimentaciones recientes e implementaciones divergentes de la red SSM introducen límites duros o acotamientos hiper-agresivos conocidos como *clamping*.[43] Al someter matemáticamente al parámetro a constreñimientos explícitos (e.g. clamp(tensor, min, max)), las secuencias truncan de manera artificial cualquier residuo o salto espurio que desborde los confines lógicos de viabilidad.[44] Estas intervenciones matemáticas son esenciales, puesto que los valores minúsculos o gigantescos pre-sintetizados en Δ y A se enviarán inevitablemente al hardware GPU, donde su mitigación definitiva requiere intervenciones ingenieriles a bajo nivel.[43]

Mitigación de Inestabilidades a Nivel de Kernel C++/CUDA

La orquestación de la estabilidad analítica descrita chocaría inútilmente con cuellos de botella monumentales de rendimiento de hardware si el modelo intentara ejecutar iteraciones secuenciales basándose puramente en las arquitecturas estándar de abstracción, como PyTorch. La iteración secuencial de pasos autorregresivos destruye la viabilidad económica al forzar lecturas constantes de los tensores inmensos de dimensión (B, L, D, N) alojados en la lenta memoria gráfica DDR central (High Bandwidth Memory, HBM).[2] En las Unidades de

Procesamiento Gráfico, el ancho de banda es la restricción definitoria; el poder teórico TFLOP se desperdicia esperando lecturas redundantes, provocando estancamiento y estrangulando los estabilizadores en el proceso.

Para habilitar la computación veloz en conjunto con su control de gradientes robusto, los investigadores de Mamba intervinieron drásticamente el modelo mediante la escritura y desarrollo de extensiones especializadas personalizadas en lenguaje de bajo nivel CUDA y C++ (selective_scan_fwd_kernel.cuh y su complemento de retropropagación selective_scan_bwd_kernel.cuh).[41] Estas bibliotecas nativas integran tres técnicas estructurales unificadoras:

Expansión de Series de Taylor y Algoritmos Expm1

En el dominio de la evaluación de gradiente en GPU, los errores de subdesbordamiento ocurren en operaciones minúsculas. Consideremos el proceso de instanciación del tensor paramétrico $\bar{B} \approx (\Delta A)^{-1}(\exp(\Delta A) - I)\Delta B$. Evaluado burdamente en tensores CUDA nativos de nivel C++, la porción subyacente $\exp(\Delta A) - 1$ perderá la mayor parte de la granularidad codificada de su mantisa para cualquier tamaño de paso diferencial Δ cercano al cero.[16] En un micro-entorno dominado por el frágil espectro del formato FP32, si el argumento se calcula en $1.0000001 - 1.0$, el hardware destrozará la resolución y retornará un truncado fatal.[28]

Para eludir estas perturbaciones sistemáticas, la biblioteca CUDA implementa aproximaciones estabilizadas de series de Taylor para forzar la compensación aritmética. En lugar de evaluar ambas operaciones aisladas, los desarrolladores explotan funciones intrínsecas acopladas del ecosistema de C++, específicamente expm1(x) y log1p(x).[16] La macro de computación subyacente evalúa analíticamente la expansión del infinito continuo compensado $1 + x + x^2/2...$ en la jerarquía intrínseca del registro de GPU, entregando un incremento hiper-preciso en $\bar{A}$ sin sufrir cancelaciones catastróficas del exponente flotante. A su vez, los bucles de desenrollado paramétrico implementan atajos condicionales (#pragma unroll e sentencias if constexpr) limitando a que, si los fragmentos operacionales Δ amenazan las paredes limítrofes numéricas, se les restringe de rebasar la lógica interna.[42]

El "Parallel Prefix Scan" Asociativo y Segment Sums

Desviar la secuencia de una operación autorregresiva a una que goce de una escalabilidad lineal veloz a través de tensores requiere evadir la maldición del cálculo estrictamente secuencial. Mamba elude esto mediante el paradigma de programación paralela de GPU, empleando heurísticas de agrupaciones vectorizadas basadas en el legendario algoritmo "Work-Efficient Parallel Prefix Scan" dictado inicialmente por Blelloch en 1990.[2]

Este algoritmo explota el postulado lógico de que la operación de suma secuencial es inherentemente asociativa.[8] De esa forma, se agrupan instantes secuenciales locales dentro del caché de altísima velocidad integrado a los Streaming Multiprocessors (SMs) del GPU, conocido como SRAM.[2] El algoritmo carga bloques simultáneos y pre-calcula subsegmentos matriciales dinámicamente antes de intercambiarlos con hilos colindantes.[8]

Sin embargo, desde la perspectiva matemática del gradiente BPTT de la red, los "Prefijos" escaneados son inherentemente secuencias operativas de *multiplicación*, derivando productos acumulativos de tensores $\bar{A}$ como $a_i \times \cdots \times a_{j-1}$.[51] Para sortear el crecimiento caótico de un producto continuado que irremediablemente causa el colapso en underflow (0.0) de los flujos de FP16 o FP32, las formulaciones recientes subyacentes de Mamba (y solidificadas enormemente en los algoritmos SSD de Mamba-2) proponen una reformulación logarítmica de la paralela asociativa.[51]

En lugar de calcular el producto burdo (*Attempt 1: Ratios of cumprods*), la matriz transforma el bloque acumulativo al campo del exponente iterativo, sumando los tensores lógicos (*Attempt 2: Differences of cumsums*) mediante primitivas vectoriales en C++ conocidas formalmente como Segment Sums (segsum).[51] Dado que $\prod \bar{A} = \exp(\sum \log(\bar{A}))$, la GPU se encuentra sumando escalares atómicos y moderados en logaritmo dentro de los registros SRAM. La suma de factores nunca excede la asfixiante delimitación del épsilon ϵ, transformando un cuello de botella logarítmicamente inestable en un flujo atómico estable, robusto, e invariante al fracaso de mantisa de la arquitectura de Von Neumann.[28]

Recomputación en Reversa (Activation Recomputation) para BPTT

Finalmente, para que un kernel alcance el Santo Grial de un entrenamiento verdaderamente eficiente desde el espectro temporal y de memoria paramétrica compartida, este no puede depender de almacenar permanentemente matrices latentes colosales.

En un sistema ingenuo, calcular los gradientes del parámetro de discretización Δ_k demanda poseer explícitamente y de manera retrospectiva el tensor del estado iterativo inmediato en caché, de tal forma que se asocie iterativamente contra $\partial \mathcal{L} / \partial h$ a través de cada peldaño microscópico k.[2] Almacenar $h_1 \ldots h_L$ durante todo el paso progresivo (Forward Pass) excedería en órdenes de magnitud el espacio de VRAM (generando colapsos de tipo Out Of Memory - OOM).[2]

La innovación radical del *Hardware-Aware Mamba Algorithm* incide en fusionar sus kernels (kernel fusion) y utilizar sistemáticamente la táctica asimétrica de la *Recomputación Continua*.[2] En el bloque C++ subyacente, durante la fase de retropropagación del paso Backward, la estructura matricial transfiere exclusivamente el estado macro de entrada HBM al dominio

rápido transitorio SRAM. Los estados intermedios latentes son re-instanciados al vuelo (*on-the-fly*) por un subsistema gemelo sub-computacional mediante una iteración cruzada inversa (*thread_reverse_data*) que cruza dy * C acoplándolo simultáneamente en tiempo y flujo reverso contra la memoria dual de registros float2, conteniendo exp(delta * A).[2]

Esta síntesis analítica previene las demoras causadas por las lentas escrituras/lecturas periféricas hacia los dominios DRAM.[2] El coste residual de efectuar un recomputo continuo resulta matemáticamente mucho menos costoso para la GPU que depender de extraer el tensor asimétrico masivo, dotando a la memoria estática un entorno puramente controlado y exento de alteraciones estocásticas.

Evolución de SSD (Mamba-2) y Estabilidad Tensorial Basada en Matrices

La profunda sensibilidad estructural del escaneo paralelo a factores empíricos como perturbaciones paramétricas en la proyección y el truncamiento forzado exhibió sus vulnerabilidades teóricas de cuellos de botella con la aparición de dimensiones de escala colosal (la restricción de mantener el dominio de estado dimensional latente restringido estúpidamente a rangos pequeños de $N = 16$ para permitir asimilaciones por *thread warps* sincrónicos).[53]

En aras de expandir esto y consolidar la estabilidad a nivel absoluto de base de hardware, la literatura científica presenció la unificación paradigmática arquitectónica en "State Space Duality" (SSD), introducida en Mamba-2. Investigadores han probado orgánicamente la formulación dual en la que una atención iterativa recurrente en la escala Mamba no es más que el espejo subestructural de las operaciones matemáticas de matrices particionadas con la rigurosidad de bloques 1-semiseparables.[54]

Traducir la secuencia a una multiplicación algorítmica de matrices asienta su beneficio supremo en que abandona la vulnerabilidad inestable del *scalar iteration* y la aloja directamente al cobijo masivo de las capacidades de los Multiplicadores de Matrices Vectorizadas Especiales (*Tensor Cores*) de las unidades aceleradoras Hopper y Ampere (las líneas Nvidia A100 / H100).[51] Con ratios de procesamiento que rebasan las centenas de TFLOPS exclusivos dedicados en BF16 para la matriz multiplicada que automáticamente ejecuta macro reducciones aditivas con fidelidad natural en acumuladores puros interconectados de FP32, la divergencia iterativa desaparece casi mágicamente.[51]

El diseño híbrido Mamba-SSD neutraliza la fragilidad y compensa de manera superlativa la erosión de precisión natural causada en su ancestro primigenio por medio del esquema escalonado *Multi-Input Multi-Output* (MIMO), y matrices proyectadas complejas con Rotary Position Embeddings (RoPE), proporcionando una aproximación numéricamente robusta, un entrenamiento resiliente, y permitiendo que la derivación secuencial abarque horizontes

operacionales sin predecir decaimientos matemáticos irreconciliables.[1]

Conclusiones sobre la Robustez Numérica

La crisis de inestabilidad matemática al interior del sistema dinámico de la arquitectura Mamba ejemplifica una interacción profunda entre la abstracción fluida de ecuaciones diferenciales subyacentes, y la brutal restricción física discreta instaurada por la naturaleza topológica fundamental de la arquitectura de microprocesadores modernos y hardware de cómputo IEEE 754 de flotantes.

El parámetro de retención temporal Δ y el tensor direccional iterativo de proyección en el núcleo de la matriz del SSM asientan en su espalda multiplicativa una cadena asintótica densa en la retropropagación en reversa. Dado que la BPTT induce inherentemente inestabilidades crecientes o colapsos nulos, el modelaje autorregresivo exige para su sostenibilidad intervenciones formales quirúrgicas simultáneas.

Mediante la imposición radical de inicializar el espectro logarítmico sub-plano, limitando orgánicamente las componentes a parámetros formales acotados ($\bar{A} \in (0, 1]$ y transformaciones asintóticas coercitivas con base logarítmica de hiper saturación no lineal como el softplus), Mamba protege sus dinámicas macroscópicas desde los cimientos analíticos.

Más crítico aún es el dominio a bajo nivel, donde este progreso paradigmático descansa por completo sobre los pilares del kernel C++ especializado de escaneo selectivo iterativo. Es la coreografía maestra interconectada con paralelismo algorítmico Blelloch logarítmico, sumas matriciales por segmento con expansión hiper precisa compensada por iteraciones estables de Expm1 de Taylor, y procesos inmersivos de retroalimentación de cálculo temporal intra-caché en reversa (Recomputation) lo que le permite burlar las amenazas asintóticas del underflow y consolidar a Mamba. Con estas herramientas combinadas de forma coherente, el modelo estabiliza magistralmente la volatilidad inherente de los gradientes recurrentes variante en el tiempo, asegurando así la convergencia de modelos en contextos abismales de longitud inexplorada.

Obras citadas

1. Mamba-3: Improved Sequence Modeling using State Space Principles - OpenReview, fecha de acceso: marzo 4, 2026, https://openreview.net/forum?id=HwCvaJOiCj
2. Mamba: Linear-Time Sequence Modeling with Selective State Spaces - arXiv, fecha de acceso: marzo 4, 2026, https://arxiv.org/pdf/2312.00752
3. Mamba architecture : A Leap Forward in Sequence Modeling | by Puneet Hegde - Medium, fecha de acceso: marzo 4, 2026, https://medium.com/@puneetthegde22/mamba-architecture-a-leap-forward-in-

sequence-modeling-370dfcbfe44a
4. Mamba for Dummies: Efficient Linear-Time LLMs Explained - Michiel Horstman - Medium, fecha de acceso: marzo 4, 2026, https://michielh.medium.com/mamba-for-dummies-linear-time-llms-explained-0d4b51efcf9f
5. I spent the last week deconstructing the math behind Mamba's "Selection Mechanism" (Delta-Gating). Here is the intuition + derivation. : r/LocalLLaMA - Reddit, fecha de acceso: marzo 4, 2026, https://www.reddit.com/r/LocalLLaMA/comments/1qf8tpd/i_spent_the_last_week_deconstructing_the_math/
6. Mamba State-Space Models Are Lyapunov-Stable Learners - arXiv.org, fecha de acceso: marzo 4, 2026, https://arxiv.org/html/2406.00209v2
7. Mamba Explained - The Gradient, fecha de acceso: marzo 4, 2026, https://thegradient.pub/mamba-explained/
8. A Visual Guide to Mamba and State Space Models - Maarten Grootendorst, fecha de acceso: marzo 4, 2026, https://www.maartengrootendorst.com/blog/mamba/
9. [D] Can someone describe how the SSM in Mamba is much different than the concepts in a GRU / LSTM Cell? : r/MachineLearning - Reddit, fecha de acceso: marzo 4, 2026, https://www.reddit.com/r/MachineLearning/comments/18iph6a/d_can_someone_describe_how_the_ssm_in_mamba_is/
10. Forward propagation of errors through time - Nicolas Zucchet, fecha de acceso: marzo 4, 2026, https://nicolaszucchet.github.io/Forward-propagation-errors-through-time/
11. 2Mamba2Furious: Linear in Complexity, Competitive in Accuracy - arXiv.org, fecha de acceso: marzo 4, 2026, https://arxiv.org/html/2602.17363v1
12. Characterization and Mitigation of Training Instabilities in Microscaling Formats, fecha de acceso: marzo 4, 2026, https://kempnerinstitute.harvard.edu/research/deeper-learning/characterization-and-mitigation-of-training-instabilities-in-microscaling-formats/
13. Mamba: Linear-Time Sequence Modeling with Selective State Spaces - Arxiv Dives, fecha de acceso: marzo 4, 2026, https://ghost.oxen.ai/mamba-linear-time-sequence-modeling-with-selective-state-spaces-arxiv-dives/
14. What Is A Mamba Model? | IBM, fecha de acceso: marzo 4, 2026, https://www.ibm.com/think/topics/mamba-model
15. AdaptPest-Net: A Task-Adaptive Network with Graph–Mamba Fusion for Multi-Scale Agricultural Pest Recognition - PMC, fecha de acceso: marzo 4, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC12732087/
16. Ophiology (or, how the Mamba architecture works) - LessWrong, fecha de acceso: marzo 4, 2026, https://www.lesswrong.com/posts/TYLQ8gAMAmpeFcwXN/ophiology-or-how-the-mamba-architecture-works
17. Rethinking the long-range dependency in Mamba/SSM and transformer models - arXiv.org, fecha de acceso: marzo 4, 2026, https://arxiv.org/html/2509.04226v1

18. Backpropagation | Brilliant Math & Science Wiki, fecha de acceso: marzo 4, 2026, https://brilliant.org/wiki/backpropagation/
19. Backpropagation and Gradients - CS231n, fecha de acceso: marzo 4, 2026, https://cs231n.stanford.edu/slides/2018/cs231n_2018_ds02.pdf
20. Backpropagation - Wikipedia, fecha de acceso: marzo 4, 2026, https://en.wikipedia.org/wiki/Backpropagation
21. Gradient Descent vs. Backpropagation: What's the Difference? - Analytics Vidhya, fecha de acceso: marzo 4, 2026, https://www.analyticsvidhya.com/blog/2023/01/gradient-descent-vs-backpropagation-whats-the-difference/
22. How does Gradient Descent and Backpropagation work together?, fecha de acceso: marzo 4, 2026, https://datascience.stackexchange.com/questions/44703/how-does-gradient-descent-and-backpropagation-work-together
23. Adding Initial Value Support to Selective Scan Forward Kernel · Issue #285 · state-spaces/mamba - GitHub, fecha de acceso: marzo 4, 2026, https://github.com/state-spaces/mamba/issues/285
24. Gradient Vanishing in Stacked Pure-Mamba Image Model (Seeking Advice) #811 - GitHub, fecha de acceso: marzo 4, 2026, https://github.com/state-spaces/mamba/issues/811
25. YOLOv11改进– Mamba ASSG (Attentive State Space Group) 注意力状态空间组 : 增强全局上下文感知CVPR 2025 - 魔改工程师, fecha de acceso: marzo 4, 2026, https://www.sylblog.xin/archives/277
26. FP64 is All You Need: Rethinking Failure Modes in Physics-Informed Neural Networks - arXiv, fecha de acceso: marzo 4, 2026, https://arxiv.org/html/2505.10949v2
27. Numerical Stability of Mixed-Precision Training on GPU Systems - Repository Home, fecha de acceso: marzo 4, 2026, https://repositories.lib.utexas.edu/items/cdc43976-6224-47d3-96e5-c578f7cf9807
28. Floating Point Precision and Its Limitations | by Umair Akbar - Medium, fecha de acceso: marzo 4, 2026, https://akbu.medium.com/floating-point-precision-and-its-limitations-cfb7247d7789
29. GPU Compute Performance Estimation: The Mathematical Foundation Behind AI Hardware Benchmarks | by Shashanka B R | Medium, fecha de acceso: marzo 4, 2026, https://medium.com/@shashanka_b_r/gpu-compute-performance-estimation-the-mathematical-foundation-behind-ai-hardware-benchmarks-7da221bcc9a4
30. Characterizing the Behavior of Training Mamba-based State Space Models on GPUs - arXiv, fecha de acceso: marzo 4, 2026, https://arxiv.org/html/2508.17679v1
31. Numerical behavior of NVIDIA tensor cores - PMC, fecha de acceso: marzo 4, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC7959640/
32. Mamba State-Space Models Can Be Strong Downstream Learners - arXiv, fecha de acceso: marzo 4, 2026, https://arxiv.org/html/2406.00209v1

33. MAMBA STATE-SPACE MODELS ARE LYAPUNOV- STABLE LEARNERS - OpenReview, fecha de acceso: marzo 4, 2026, https://openreview.net/pdf?id=XDfyJqlB4T
34. exp the log of A? · Issue #197 · state-spaces/mamba - GitHub, fecha de acceso: marzo 4, 2026, https://github.com/state-spaces/mamba/issues/197
35. Mamba Modulation On the Length Generalization of Mamba - arXiv, fecha de acceso: marzo 4, 2026, https://arxiv.org/html/2509.19633v1
36. Mamba Modulation On the Length Generalization of Mamba - OpenReview, fecha de acceso: marzo 4, 2026, https://openreview.net/pdf?id=QEU047bE8p
37. Along comes a Mamba: an evolution in sequence models based on state space models | by Wilbur de Souza | Medium, fecha de acceso: marzo 4, 2026, https://medium.com/@wilburdes/along-comes-a-mamba-an-evolution-in-sequence-models-based-on-state-space-models-2bd3d0e02d86
38. Here Comes Mamba: The Selective State Space Model | Towards Data Science, fecha de acceso: marzo 4, 2026, https://towardsdatascience.com/here-comes-mamba-the-selective-state-space-model-435e5d17a451/
39. Block-Biased Mamba for Long-Range Sequence Processing - arXiv, fecha de acceso: marzo 4, 2026, https://arxiv.org/html/2505.09022v1
40. Upload folder using huggingface_hub · FengheTan9/MambaMIM at 6da2a44, fecha de acceso: marzo 4, 2026, https://huggingface.co/FengheTan9/MambaMIM/commit/6da2a445da659f029345ddb97e3de129085b8904
41. selective_scan_cuda · Issue #605 · state-spaces/mamba - GitHub, fecha de acceso: marzo 4, 2026, https://github.com/state-spaces/mamba/issues/605
42. Bug in selective scan backward · Issue #368 · state-spaces/mamba - GitHub, fecha de acceso: marzo 4, 2026, https://github.com/state-spaces/mamba/issues/368
43. Selective Synchronization Attention - arXiv.org, fecha de acceso: marzo 4, 2026, https://arxiv.org/html/2602.14445v1
44. Parallelizable Neural Turing Machines - arXiv, fecha de acceso: marzo 4, 2026, https://arxiv.org/html/2602.18508v1
45. frontier model training methodologies - Alex Wa's Blog, fecha de acceso: marzo 4, 2026, https://djdumpling.github.io/2026/01/31/frontier_training.html
46. RWKV-7 "Goose" with Expressive Dynamic State Evolution - OpenReview, fecha de acceso: marzo 4, 2026, https://openreview.net/pdf?id=ayB1PACN5j
47. selective-scan/selective_scan_fwd_kernel.cuh · kernels-community/mamba-ssm at 85e64a572bbe3c30c688b94fb0590dc07b9e9d00 - Hugging Face, fecha de acceso: marzo 4, 2026, https://huggingface.co/kernels-community/mamba-ssm/blob/85e64a572bbe3c30c688b94fb0590dc07b9e9d00/selective-scan/selective_scan_fwd_kernel.cuh
48. Error-Free Linear Attention is a Free Lunch: Exact Solution from Continuous-Time Dynamics - arXiv, fecha de acceso: marzo 4, 2026, https://arxiv.org/pdf/2512.12602
49. Possible Rounding/Precision Errors in CUDA Math APIs? - NVIDIA Developer Forums, fecha de acceso: marzo 4, 2026,

https://forums.developer.nvidia.com/t/possible-rounding-precision-errors-in-cuda-math-apis/301707
50. Mamba No. 5 (A Little Bit Of...) - Sparse Notes, fecha de acceso: marzo 4, 2026, https://jameschen.io/jekyll/update/2024/02/12/mamba.html
51. State Space Duality (Mamba-2) Part III - The Algorithm | Tri Dao, fecha de acceso: marzo 4, 2026, https://tridao.me/blog/2024/mamba2-part3-algorithm/
52. State Space Duality (Mamba-2) Part III - The Algorithm | Goomba Lab, fecha de acceso: marzo 4, 2026, https://goombalab.github.io/blog/2024/mamba2-part3-algorithm/
53. Mamba2: The Hardware-Algorithm Co-Design That Unified Attention and State Space Models | by Daniel Stallworth | Medium, fecha de acceso: marzo 4, 2026, https://medium.com/@danieljsmit/mamba2-the-hardware-algorithm-co-design-that-unified-attention-and-state-space-models-77856d2ac4f4
54. Mamba-2: Algorithms and Systems | Princeton Language and Intelligence, fecha de acceso: marzo 4, 2026, https://pli.princeton.edu/blog/2024/mamba-2-algorithms-and-systems
55. Show HN: Mamba3-minimal – PyTorch implementation of Mamba-3 | Hacker News, fecha de acceso: marzo 4, 2026, https://news.ycombinator.com/item?id=47154922